



engineered for ambition

A PostgreSQL Upgrade Journey

Who Are we



Huseyin Demir

PostgreSQL Database Engineer



Milen Blagojevic

PostgreSQL Database Engineer

Agenda

PostgreSQL Upgrade Journey

01

The Reasons

Why and how we made the strategic choice to upgrade our PostgreSQL setup.

02

The Approach

The execution phase and high-scale rollout patterns we implemented.

03

Incidents

Problems, key failures and lessons learned during the process.

04

The Rewards

The post-upgrade results and engineering performance gains.



engineered for ambition

PHASE 01

The Reason

The Reasons Behind the Decision

Staying on the legacy version meant sacrificing essential performance and operational resilience.



End of Life Announcements

The current PostgreSQL version was approaching the end of its support window.



Performance Ceilings

Staying meant giving up meaningful improvements in vacuum performance and WAL throughput.



New Features

More than one new feature which could help our workload.

Performance Ceilings



The primary modern performance bottlenecks identified in our upgrade evaluation.



MOTIVATOR 01

Autovacuum & Vacuum



MOTIVATOR 02

Transactional Locks

Pre-PG 17 Autovacuum: The Multi-Pass Problem

How restricted memory budgets force redundant index scans on bloated tables

Step 1: Heap Scan & TID Collection

Scanning Table & Collecting TIDs

Vacuum scans the heap, collecting dead tuple IDs (TIDs) into a flat array. Memory fills up rapidly toward the **maintenance_work_mem** limit (98% full).

Step 2: Array Full & Pause

Forced Full Index Scans

When memory is completely exhausted, the heap scan is paused. Autovacuum is forced to perform a **full scan of all indexes** to clear the collected TIDs.

Step 3: Resume Heap Scan

Collecting the Next Batch

Once index scans finish, memory is cleared. Vacuum resumes the heap scan, starting a second pass to collect the remaining batch of dead TIDs.

Step 4: Repeated Index Scans

Massive I/O Overhead

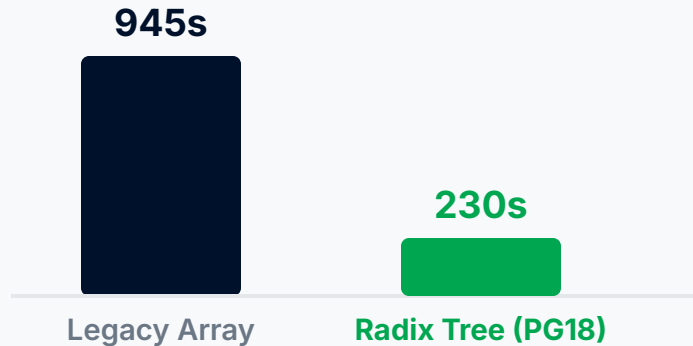
Memory fills up again, triggering another round of full index scans. This **multi-pass behavior** results in 2x table & 2x index scans, wasting massive I/O capacity.

The Autovacuum Duration

The Problem The legacy dead tuple array exhausted memory budgets quickly on 100TB+ clusters, forcing multiple index sweeps.

The Solution PostgreSQL 17+ introduced a compact radix tree-based structure for the TID store.

Autovacuum Duration (Shorter is Better)



4.1x

Reduction

The Impact: A massive drop in wall-clock time, allowing autovacuum processes to comfortably keep pace with sustained writes on heavily bloated tables.

The Fast-Path Lock Ceiling

FIXED IN PG 18

MAX_LOCKS_PER_TRANSACTION

WHAT IT CONTROLS

Sizes the shared lock table:

```
max_locks_per_transaction × (max_connections +  
max_prepared_transactions)
```

THE HIDDEN LIMIT

Weak locks (**SELECT-type**) use a private per-backend **fast path** — fixed at **16 slots** through PG17, no matter how high you set the GUC. Every index on a table needs one too.

THE FIX

PG18 sizes the fast-path array **from max_locks_per_transaction itself**, in groups of 16 — the setting you already tune now actually relieves lock-manager contention, not just table capacity.

16 → 64

Default fast-path slots per backend

PG13–17: fixed at **16**, always • PG18: **64** by default, scales with the GUC

Table with 1 PK + 20 indexes = 22 locks/query

PG17 (16 slots) **6 locks overflow to shared table, every call**

PG18 (64 slots) all 22 stay fast-path

Mechanism, not a benchmark • No published pgbench comparison exists yet for this change — the win is structural (fewer LWLock:LockManager acquisitions). Source: PG18 release notes; "Sixteen Locks Ought to Be Enough for Anybody," thebuild.com.

Fast-Path Capacity: PG17 vs. PG18

MAX_LOCKS_PER_TRANSACTION

■ used by this query ■ free capacity ■ forced to shared lock table

PG13 – PG17

fast path fixed at 16 slots / backend

Query needs 22 locks (1 table + 21 indexes)



+6 locks can't fit — overflow



Shared Lock Table

partitioned · LWLock:LockManager contention on every call

PG18

fast path scales with the GUC — 64 slots by default

Query needs 22 locks (1 table + 21 indexes)



all 22 fit — 42 slots free



Shared Lock Table

not touched by this query

Illustrative · based on the documented PG18 change (fast-path array now sized from `max_locks_per_transaction`, in groups of 16, vs. a fixed 16 through PG17). No published pgbench comparison exists yet for this specific change.
Source: PG18 release notes; "Sixteen Locks Ought to Be Enough for Anybody," thebuild.com.

Removing the ProcArrayLock Bottleneck

PG14+

WHAT IT DOES

Called on every query to build the MVCC snapshot — "**who else is running right now?**"

THE BOTTLENECK

Every call scanned the full proc array under **ProcArrayLock**, with backends also writing xmin into the same hot cache line — cost grew with connection count, even mostly-idle ones.

THE OPTIMIZATION (THE FIX)

Removed the cache-line write/read collision, packed hot fields densely, and **cached snapshots** when nothing had changed since the last call.

2.17×

Throughput Scaling

At 5,000 connections

Legacy: **732K TPS**

PG14: **1.59M TPS**

2.06×

Idle Connection Scaling

10,000 idle + 1 active connection

Legacy: **16K TPS**

PG14: **33K TPS**

Benchmark Setup • pgbench, read-only, PostgreSQL 13 vs. 14 (patched) — A. Freund, "Improving connection scalability: GetSnapshotData()," pgsql-hackers, 2020.

New Features

Key performance, replication, and maintenance enhancements in PostgreSQL



PG 17+

Replication slots failover

PostgreSQL 17+ natively synchronizes logical replication slots to standbys.



PG 18+

Asynchronous IO

PostgreSQL 18+ AIO subsystem that can improve performance of sequential scans, bitmap heap scans, vacuums, and other operations.



PG 14+

Reindex

Since PostgreSQL 14 `CREATE INDEX CONCURRENTLY` and `REINDEX CONCURRENTLY` no longer limit the dead row removal of other relations.



PG 15+

Recovery Prefetching

Setting `maintenance_io_concurrency` issues hints to prefetch upcoming blocks during WAL replay.

SQL-Queryable System Views



Enable fleet-wide monitoring and diagnostics without host-level tooling



pg_stat_wal

Exposes WAL records written, bytes written, and sync durations directly via SQL queries.



pg_stat_io

Breaks down I/O by backend type and context, allowing us to track autovacuum impact accurately.



pg_stat_subscription_stats

Adds explicit apply and sync error counters to streamline logical subscription monitoring.

Fleet Visibility Benefit

Turns complex, unstructured host-level diagnostics into simple, structured threshold queries we can scale and alert on fleet-wide.



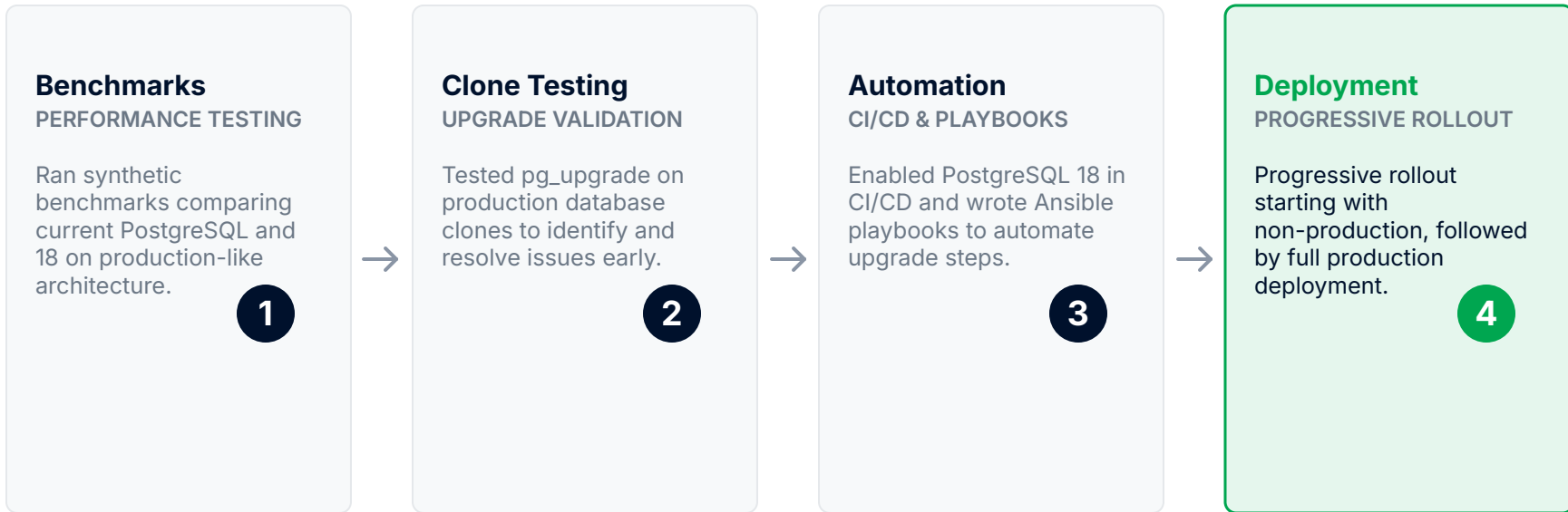
engineered for ambition

PHASE 02

Upgrade Approach

Upgrading Safely

Our structured approach to ensuring a safe, automated, and predictable upgrade path across all environments.



Are we upgrading only PostgreSQL ?



Key areas that complicate scaling and introduce upgrade complexity

Replication Topology

pg_upgrade only operates on the primary; secondaries require terabytes of rsyncing.

Logical Slots

pg_upgrade drops logical slots, requiring a manual disable, recreate and re-enable workflow.

Physical Slots

Explicitly named physical slots must be captured and manually recreated post-upgrade.

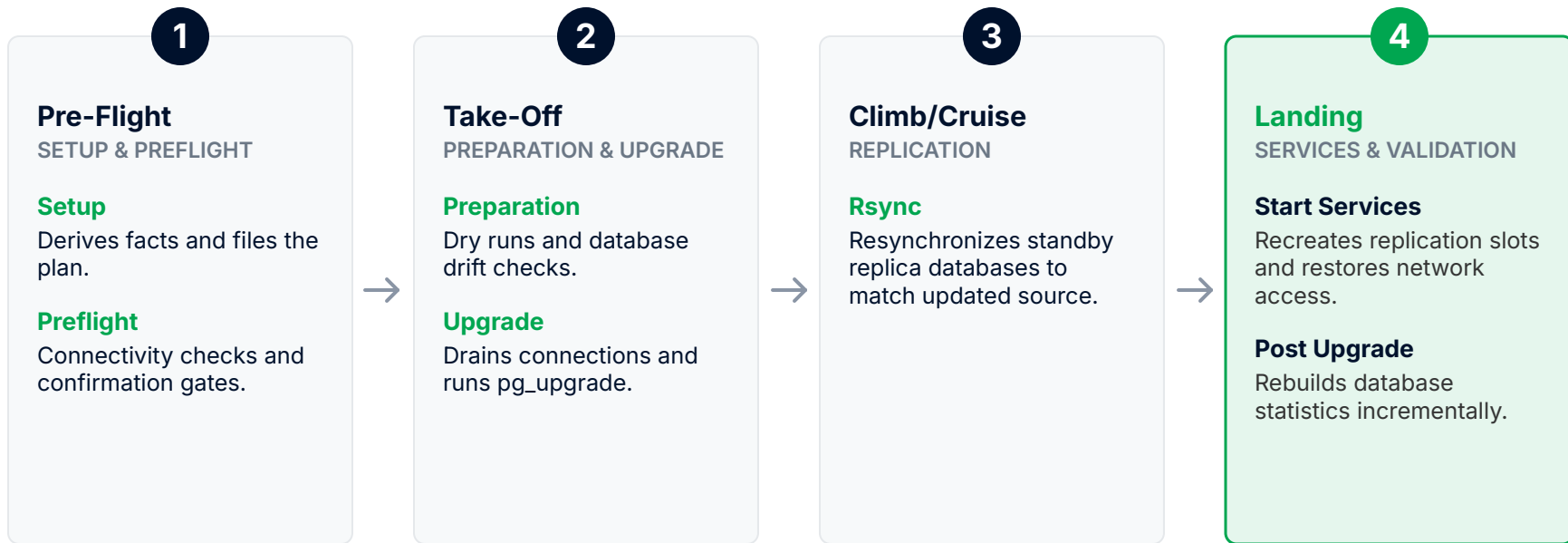
Extensions & Backups

Target-version extensions (e.g., pg_failover_slots) must be present, and Barman backup paths must be updated.



Upgrading with the Approach

Four independent, resumable Ansible phases mapped to a flight



Automated, phased rollouts achieved massive upgrades with minimal downtime.

"Upgrading PostgreSQL cluster with primary and 4 replicas database caused a downtime of less than 5 minutes."

- **Risk Reduction:** The phased approach eliminated heroics and reduced operational risk across the fleet.
- **Future-Proofing:** We eliminated legacy scalability ceilings and secured a more efficient data layer.
- **Repeatability:** The playbook allows for safe, repeatable automated rollouts across hundreds of clusters.

PHASE 03

The Post-Mortem We Didn't Plan For

"Because 'everything worked in staging' is the ultimate famous last words."

Orphaned Privileges in pg_init_privs

a dropped role, a silent catalog leftover, and a restore that fails mid-upgrade

The Setup

weeks before the upgrade, unrelated cleanup

```
CREATE EXTENSION some_extension;  
owned by role: extension_owner  
pg_init_privs row created  
ACL stores extension_owner's OID  
ALTER DATABASE ... RENAME TO ...;  
DROP ROLE extension_owner;  
pg_init_privs row not cleaned up - orphaned
```

Looks clean — a role was dropped, nothing else touched

The Upgrade

pg_upgrade --check passes; the restore step doesn't

```
$ pg_upgrade --check  
Clusters are compatible ✓  
  
$ pg_upgrade  
pg_dump | pg_restore (old → new)  
  
ERROR: role "extension_owner" does not exist  
while restoring initial privileges
```

Workaround: purge the orphan row, then pg_upgrade succeeds

Root cause · `DROP ROLE` doesn't clean up the role's OID embedded inside `pg_init_privs`' stored ACL arrays — and `pg_upgrade --check` never inspects them.
Patch submitted · proposed for backpatch to all supported versions — `pg_dump` excludes `pg_init_privs` entries whose grantee role no longer exists, instead of emitting a `GRANT` to a role that's gone.

The Ghost in pg_init_privs

a catalog row that outlives the role it points to

While the role exists

the ACL entry has something real to point at

pg_authid			pg_init_privs	
rolname	oid		objoid	initprivs
extension_owner	24601	OID match	16482	{extension_owner=U/...}
app_reader ...			16483 ...	



(sleeping quietly for now)

reference intact — nothing to see here

After DROP ROLE

the ACL entry is now haunting an empty seat

pg_authid			pg_init_privs	
rolname	oid		objoid	initprivs
extension_owner	gone	no match	16482	{extension_owner=U/...}
app_reader ...			16483 ...	



OID 24601

still here! nobody dropped ME 🗿

orphan record — points at an OID nothing owns anymore

What happens next · `pg_upgrade` 's `pg_dump` / `pg_restore` step tries to turn this ACL back into a real `GRANT ... TO extension_owner` — and fails, because `pg_upgrade --check` never looked at this table.

Nested Custom Tablespaces

The Setup

custom tablespace created inside the PG13 data directory

```
/var/lib/postgresql/  
├── 13/data/ PostgreSQL 13 — $PGDATA  
│   └── base/ ...  
│       └── our_custom_tablespace/ ⚠ real storage  
└── 14/data/ PostgreSQL 14 — $PGDATA  
    └── pg_tblspc/16401 → nested in 13/data/
```

Working — but the tablespace lives inside PG13's directory

The Cleanup

`rm -rf /var/lib/postgresql/13/data` to reclaim disk space

```
/var/lib/postgresql/  
├── 13/data/ X DELETED  
│   └── our_custom_tablespace/ X DELETED  
└── 14/data/ PostgreSQL 14 — $PGDATA  
    └── pg_tblspc/16401 — X → target no longer exists
```

PG14's tablespace is now unreadable

Root cause · the tablespace's real storage was nested inside \$PGDATA of the old cluster instead of a version-agnostic path.

Fix · relocate custom tablespaces outside any cluster's data directory before upgrading — old or new.

Constraint Name Collision in pg_upgrade

a custom CHECK constraint that happens to share PG17's new auto-generated name

The Collision

PG17 starts auto-naming NOT NULL constraints

```
-- PG13: table has a named CHECK constraint
CONSTRAINT orders_customer_id_not_null
CHECK (customer_id IS NOT NULL)

-- PG17+: NOT NULL is now a real pg_constraint
row

auto-generated as: {table}_{column}_not_null
orders_customer_id_not_null identical name
```

Nobody touched this table in years it just happens to collide

The Failure and the Fix

pg_upgrade dies on a duplicate key; renaming first avoids it

```
$ pg_upgrade
ERROR: duplicate key value violates
unique constraint "pg_constraint_conname..."

-- fix: rename the old CHECK before upgrading
ALTER TABLE orders RENAME CONSTRAINT
orders_customer_id_not_null
TO orders_customer_id_check;
```

Fix applied pre-upgrade; drop the redundant CHECK afterward

Root cause starting in PG17, NOT NULL constraints are materialized in pg_constraint (type 'n') and auto-named {table}_{column}_not_null during upgrade colliding with any pre-existing constraint that shares that name. **Fix** rename the conflicting constraint before running pg_upgrade; drop the redundant CHECK once complete.

Three Incidents on the Way to PG18

what `pg_upgrade --check` didn't catch

01

Nested Custom Tablespace

WHAT HAPPENED

Tablespace storage physically lived inside PG13's

`$PGDATA`

ROOT CAUSE

Post-upgrade cleanup ran `rm -rf` on the old data directory

FIX

Relocate tablespaces outside any cluster's data directory

02

Orphaned `pg_init_privs` Row

WHAT HAPPENED

`DROP ROLE` left a dangling ACL entry behind

ROOT CAUSE

`pg_upgrade --check` never inspects

`pg_init_privs`

FIX

Patch submitted — `pg_dump` skips orphaned entries (proposed backpatch)

03

Constraint Name Collision

WHAT HAPPENED

A legacy CHECK constraint collided with PG17's new auto-generated name

ROOT CAUSE

PG17 materializes NOT NULL constraints in

`pg_constraint` with predictable names

FIX

Rename the constraint first; filed as Bug #19393

None of these showed up in `pg_upgrade --check` — they surfaced in dump/restore, or after the upgrade was already considered done.



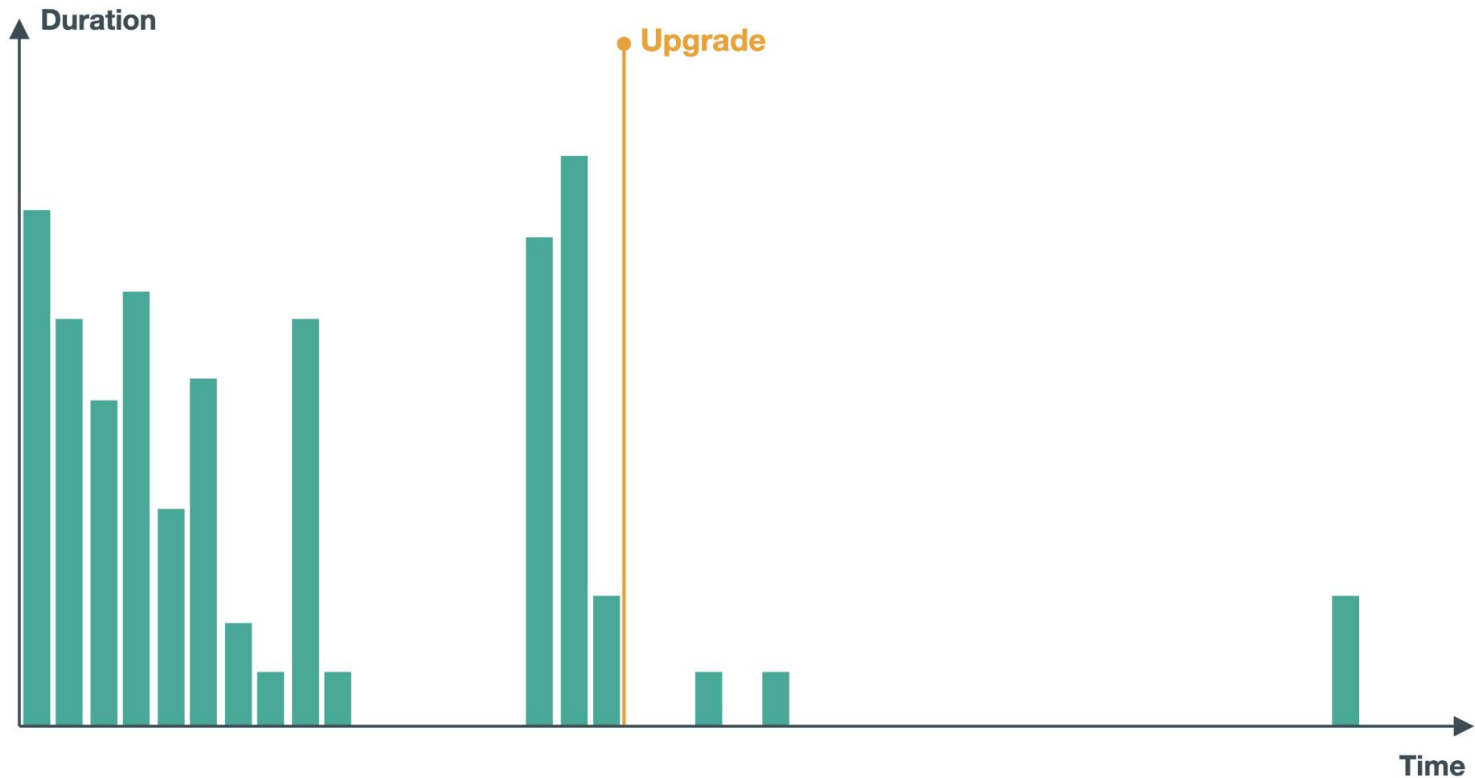
engineered for ambition

PHASE 04

Post Upgrade Results

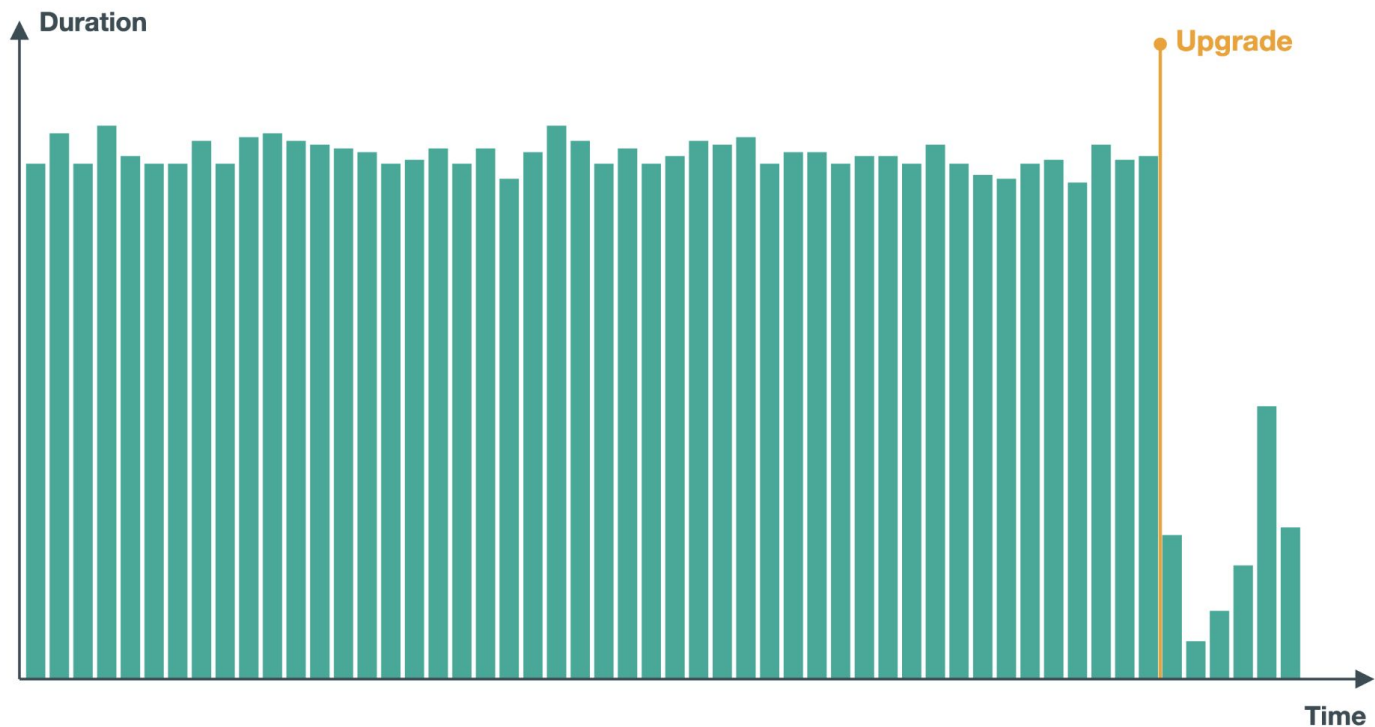
Post-Upgrade Impact Analysis

Drastic reduction in daily long-running queries (LRQ) following the July 15, 2026 upgrade



Post-Upgrade Performance Impact

Drastic reduction in the median duration of postgres maintenance tasks following the July 20, 2026 upgrade



Autovacuum Performance Comparison

Production telemetry shows massive efficiency gains with PG 18 upgrades

METRIC	Current PG Version	Current PG Version	PG 18	RESULT & ANALYSIS
DURATION Elapsed Time	0.60s	0.46s	0.29s	-51.7% PG 18 autovacuum finished significantly faster, demonstrating smooth, highly efficient background operations.
I/O IMPACT Pages Dirtied	1,313	788	92	-92.99% Far fewer modified pages drastically reduces background write pressure, lowering disk wear and system load.
WRITE PATH WAL Records	10,499	9,858	8,500	-19.0% Lower WAL record count indicates less maintenance churn.



engineered for ambition

Thank you